4.5 Deep recurrent neural networks

- up to now, we only discussed RNNs with a single hidden layer
- in it, the specific functional form of how latent variables and observations interact is rather arbitrary
- this is not a big problem, as long as we have enough flexibility to model different types of interactions
- with a single layer, however, this can be quite challenging
- in the case of the linear models, we fixed this problem by adding more layers
- within RNNs, this is a bit trickier, since we first need to decide how and where to add extra nonlinearity

4.5 Deep recurrent neural networks

- in fact, we could stack multiple layers of RNNs on top of each other
- this results in a flexible mechanism, due to the combination of several simple layers
- in particular, data might be relevant at different levels of the stack
- for instance, we might want to keep high-level data about financial market conditions (bear or bull market) available, whereas, at a lower level, we only record shorter-term temporal dynamics
- beyond all the above abstract discussion, it is probably easiest to understand the family of models we are interested in by reviewing Figure 12; it describes a deep RNN with L hidden layers
- each hidden state is continuously passed to both the next time step of the current layer, and the current time step of the next layer

4.5 Deep recurrent neural networks

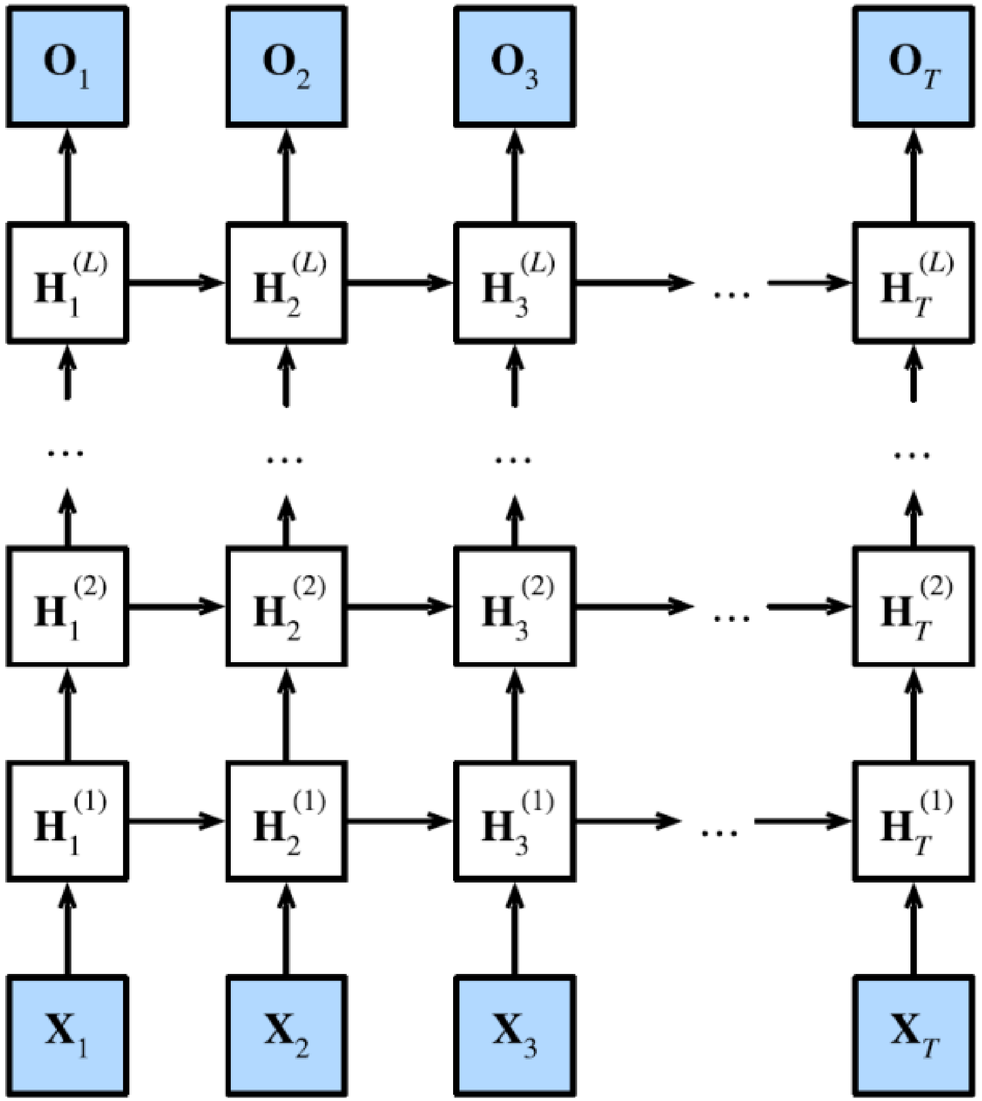


Figure 12: Architecture of a deep RNN.

4.5 Deep recurrent neural networks

- we can formalize the functional dependencies within the deep architecture of L hidden layers depicted in Figure 12
- our following discussion focuses primarily on the vanilla RNN model, but it applies to other sequence models, too
- suppose that we have a mini-batch input $\boldsymbol{X}_t \in \mathbb{R}^{n \times d}$ (number of examples: n, number of inputs in each example: d) at time step t
- at the same time step, let the hidden state of the lth hidden layer ($l = 1, \ldots, L$) be $\boldsymbol{H}_t^{(l)} \in \mathbb{R}^{n \times h}$ (number of hidden units: h) and the output layer variable be $\boldsymbol{O}_t \in \mathbb{R}^{n \times q}$ (number of outputs: q)
- setting $\boldsymbol{H}_t^{(0)} = \boldsymbol{X}_t$, the hidden state of the lth hidden layer that uses the activation function ϕ_l is expressed as follows:

$$\boldsymbol{H}_t^{(l)} = \phi_l(\boldsymbol{H}_t^{(l-1)} \boldsymbol{W}_{xh}^{(l)} + \boldsymbol{H}_{t-1}^{(l)} \boldsymbol{W}_{hh}^{(l)} + \boldsymbol{b}_h^{(l)}), \tag{6}$$

where the weights $\boldsymbol{W}_{xh}^{(l)} \in \mathbb{R}^{h \times h}$ and $\boldsymbol{W}_{hh}^{(l)} \in \mathbb{R}^{h \times h}$, together with the bias $\boldsymbol{b}_h^{(l)} \in \mathbb{R}^{1 \times h}$, are the model parameters of the lth hidden layer

4.5 Deep recurrent neural networks

- in the end, the calculation of the output layer is only based on the hidden state of the final Lth hidden layer:

$$\boldsymbol{O}_t = \boldsymbol{H}_t^{(L)} \boldsymbol{W}_{hq} + \boldsymbol{b}_q,$$

  where the weight $\boldsymbol{W}_{hq} \in \mathbb{R}^{h \times q}$ and the bias $\boldsymbol{b}_q \in \mathbb{R}^{1 \times q}$ are the model parameters of the output layer
- just as with MLPs, the number of hidden layers L and the number of hidden units h are hyperparameters
- in other words, they can be tuned or specified by us
- in addition, we can easily get a deep gated RNN by replacing the hidden state computation in (6) with that from a GRU or an LSTM